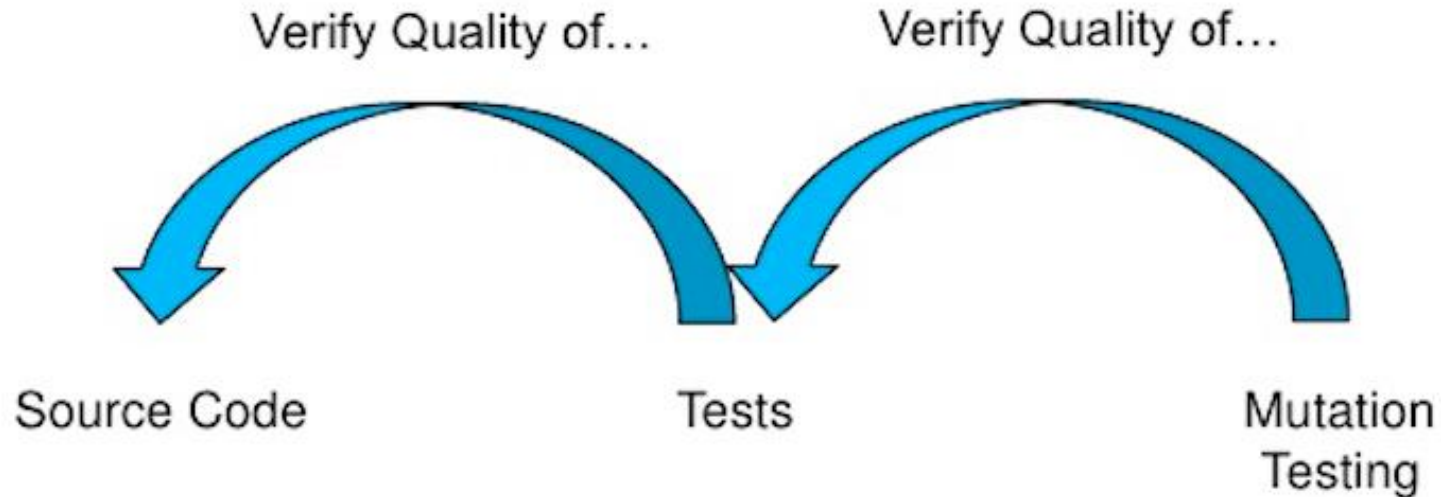


Main Findings from My Research on Mutation Testing

João Paulo Diniz

LabSoft Seminar. Sep 24th, 2026

Introduction



Example of a mutant

Original:

```
public class Taxes {  
  
    double simpleTax(double amount) {  
  
        return amount * 0.2;  
    }  
}
```

Example of a mutant

Mutation place:

```
public class Taxes {  
  
    double simpleTax(double amount) {  
  
        return amount * 0.2;  
    }  
}
```

Example of a mutant

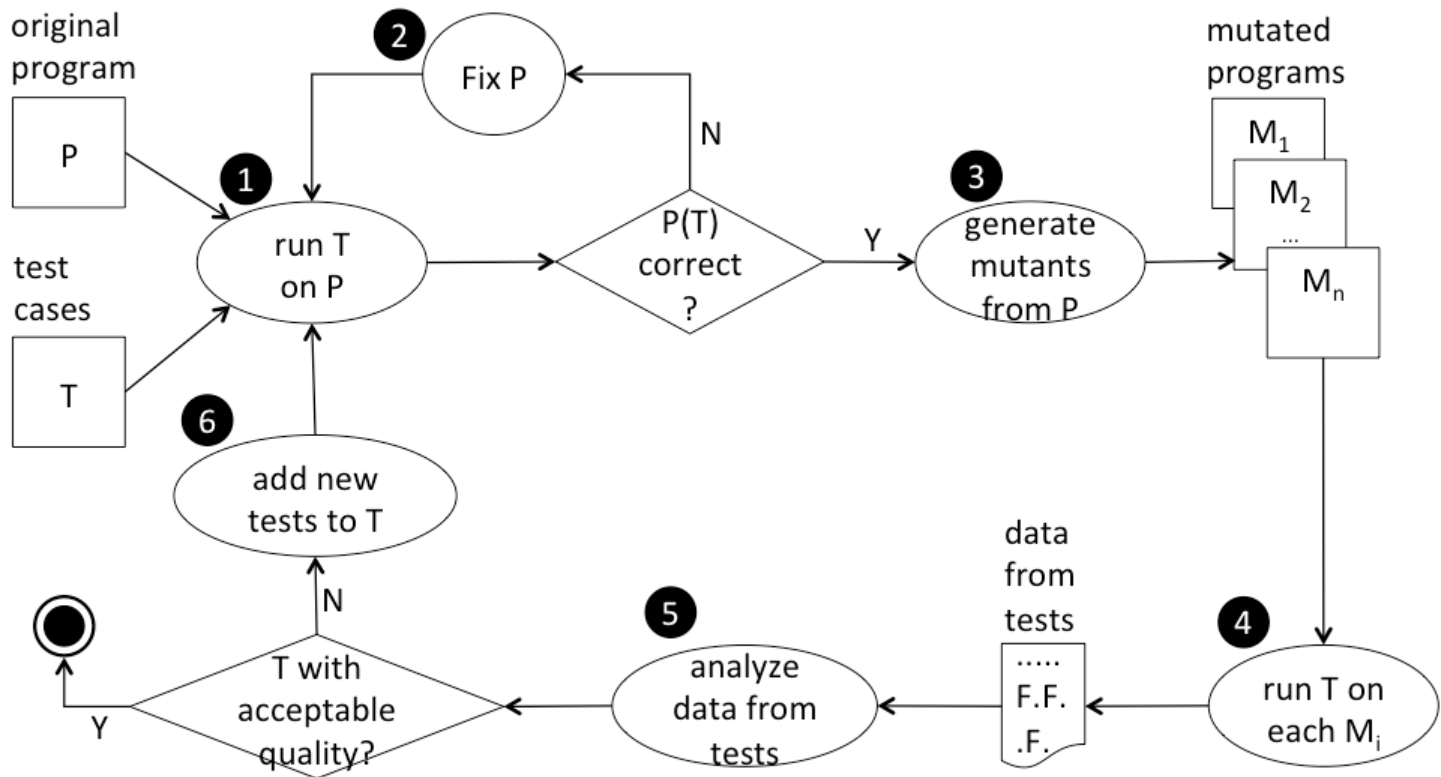
* → +

```
public class Taxes {  
  
    double simpleTax(double amount) {  
  
        return amount + 0.2;  
    }  
}
```

Mutation Testing

- ❑ Introducing **artificial changes** into original source code
 - Syntactic changes (**mutations**)
 - Representing real common programming mistakes
 - Changed programs are called **mutants**
- ❑ Running test cases on mutants
 - Result different from original: mutant **killed**
 - Otherwise: **alive**

Mutation testing process



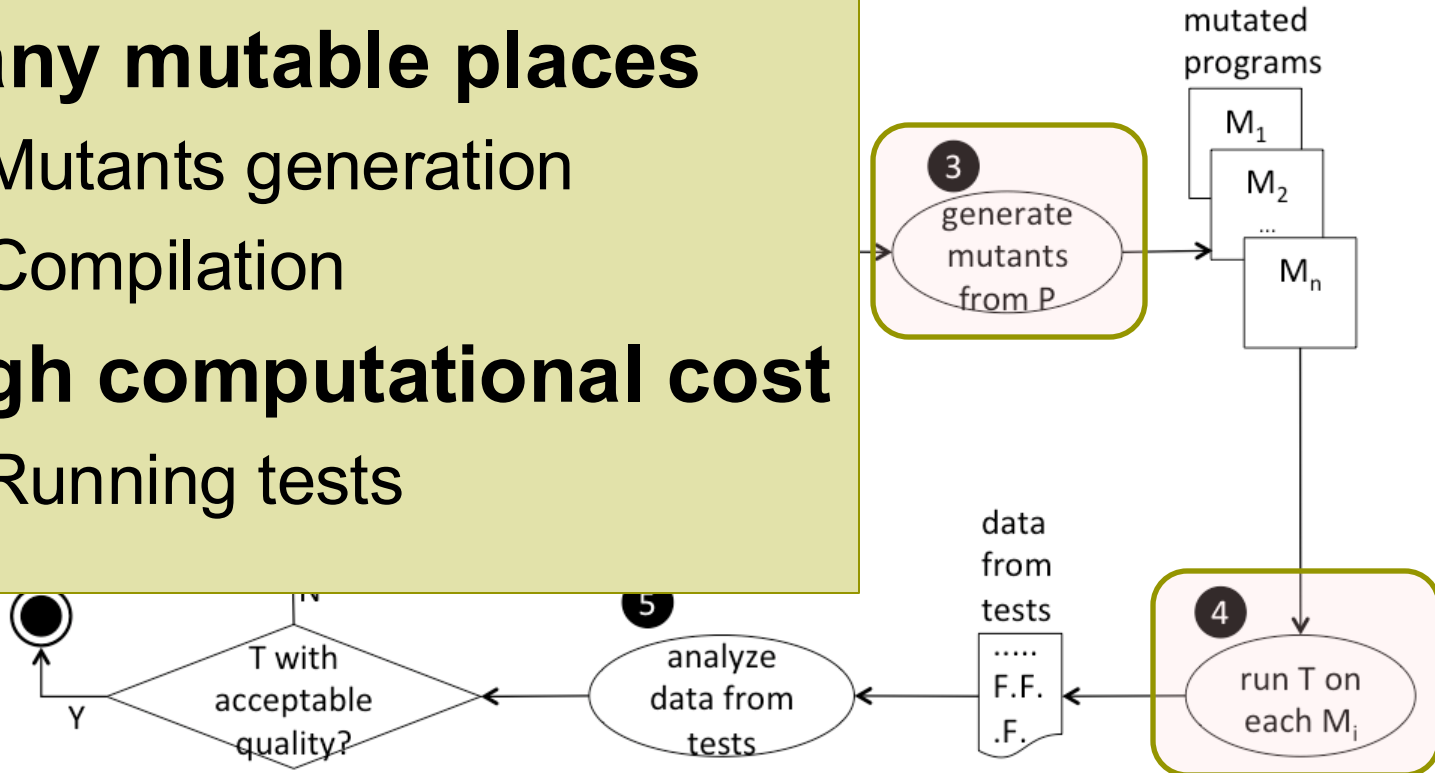
Mutation testing drawbacks

3. Many mutable places

- Mutants generation
- Compilation

4. High computational cost

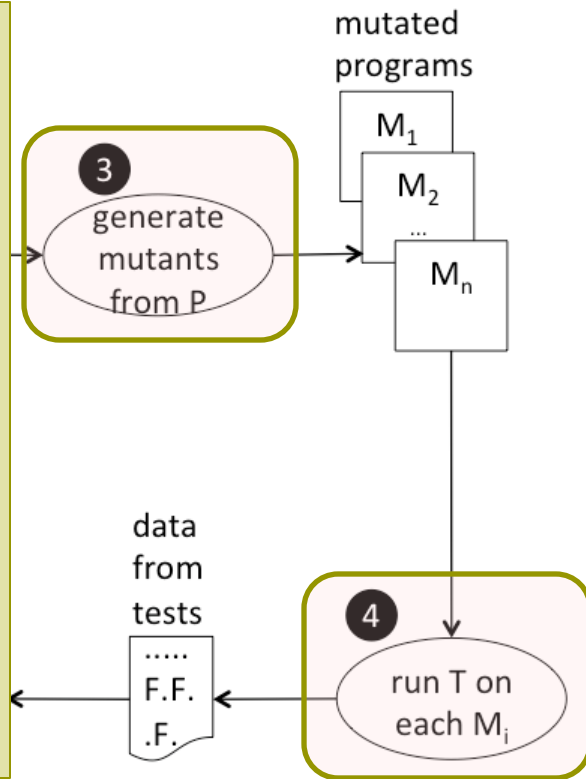
- Running tests



Mutation testing drawbacks

❑ Cost reduction techniques

- Number of mutants
- Number of test cases
- Test case prioritization
- **Higher-order mutation testing**



Higher-order Mutants

Higher-order mutants

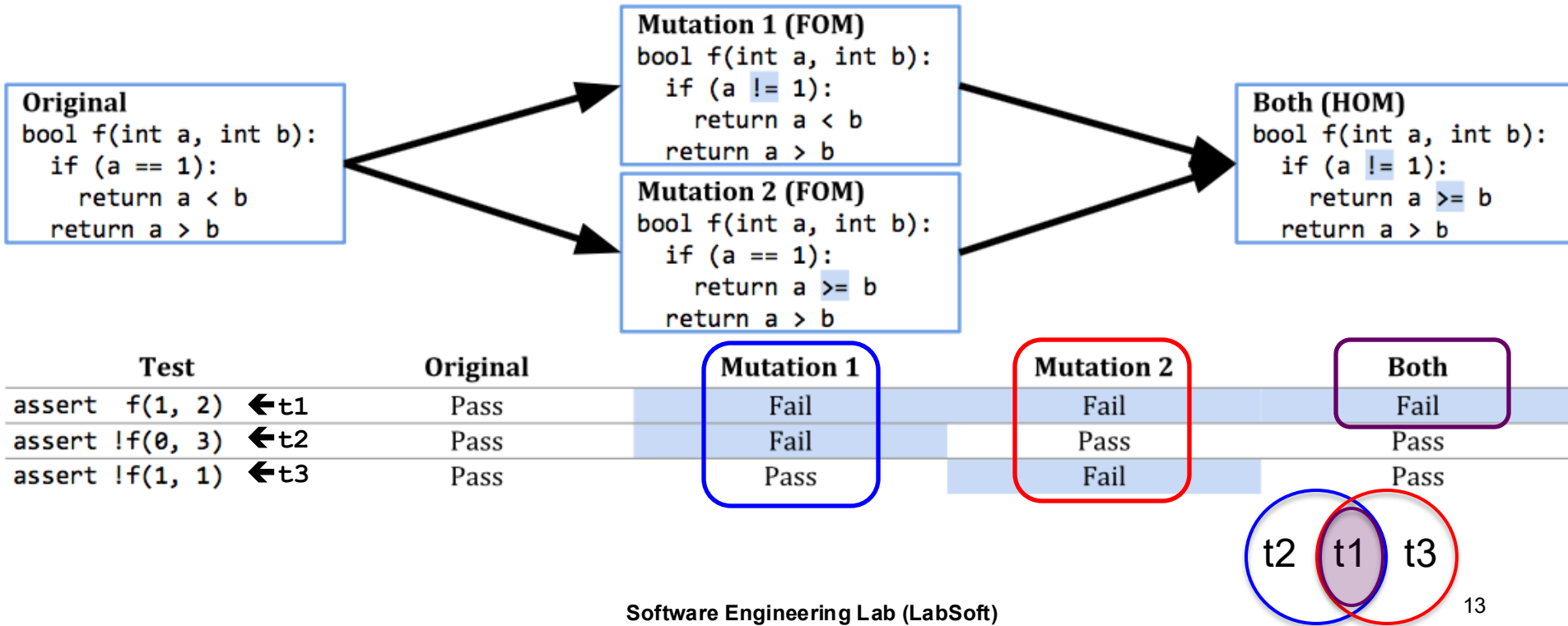
- 1 mutation → first-order mutant (**FOM**)
- > 1 mutations → higher-order mutant (**HOM**)
 - 2 mutations → 2nd order mutant (**2OM**)

- **n** mutations can generate
 - $\sim C_2^n$ 2OMs
 - $\sim C_3^n$ 3OMs
 - $\sim 2^n$ HOMs (exponential)

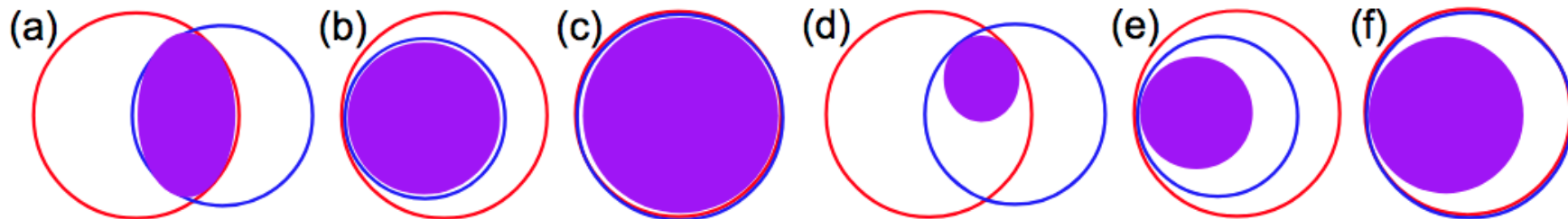
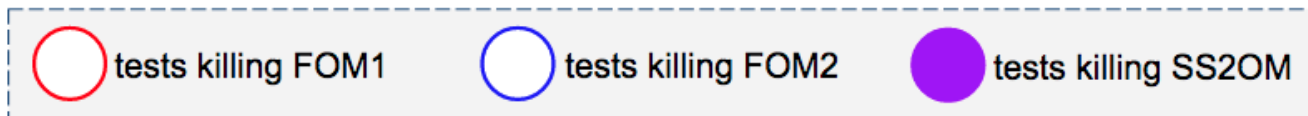
Higher-order mutants

- Most are **more likely to be killed** than FOMs
- But few are **harder** to be killed
 - are **more representative**
 - represent **subtler faults**
 - mutations can (partially) **mask** each other

A 2OM subsumption example



Venn Diagram “shapes”



SSHOMs research

- ❑ Exponential search space
- ❑ Dynamic search (running test suite)
- ❑ Trends:
 - Find as many SSHOMs as possible
 - Evolutionary algorithms
 - ❑ Genetic Algorithms
 - ❑ Hill Climbing
 - Multi-objective search

Common Setup and Tooling

Dataset - Java subject systems

System	Version	LOC	# Tests	JUnit
Vending Machine	Exceptions	~100	35	4
Triangle	n/a	34	12	4
Monopoly	n/a	1,181	124	3
Commons CSV	1.8	~2k	325	4
Commons CLI	1.4	2,699	318	4
ECal	2003.10	3,626	224	3
Commons Validator	1.6	7,409	536	4
Gson	2.9.0	> 10k	1,089	3 and 4
Chess	n/a	4,924	930	3 and 4
Joda-Time	2.12.7	~30k	4,232	4

Mutation operators implemented

Operator	Name	Examples	
		Original	Mutants
AOR	Arithmetic Operator Replacement	<code>a + b</code>	<code>a - b</code> <code>a * b</code> <code>a / b</code> <code>a % b</code>
ROR	Relational Operator Replacement	<code>a < b</code>	<code>a > b</code> <code>a <= b</code> <code>a >= b</code> <code>a == b</code> <code>a != b</code>
LCR	Logical Connector Replacement	<code>a b</code>	<code>a && b</code>
SBR	Statement Block Removal	<code>a = b+1;</code> <code>if (...) {</code> <code>...</code> <code>}</code>	<code>∅</code> <code>∅</code>

FOMs generated

System	AOR	LCR	ROR	SBR	Total
Vending Machine	4	1	40	12	57
Triangle	36	7	85	10	138
Monopoly	120	8	380	358	866
Commons CSV	208	56	250	411	925
Commons CLI	108	80	355	539	1,082
ECal	136	9	315	753	1,207
Commons Validator	324	174	1,675	1,024	3,197
Gson	536	211	1,430	1,535	3,712
Chess	1,876	244	1,890	1,309	5,287
Joda-Time	4,300	437	6,985	4,701	16,333
Overall	7,648	1,196	13,315	10,645	32,804

SS2OMs Characterization

Research questions

- ❑ **RQ2:** ❑ What is the **distribution** of the mutations in the analyzed Valuable 2OMs?
- ❑ **RQ3:** What are the **most common killing tests** Venn Diagram “shapes” of the Valuable 2OMs found?
- ❑ **RQ6:** What is the **reduction in the number of mutants** that SS2OMs achieve by replacing their constituent FOMs?

SS2OMs found

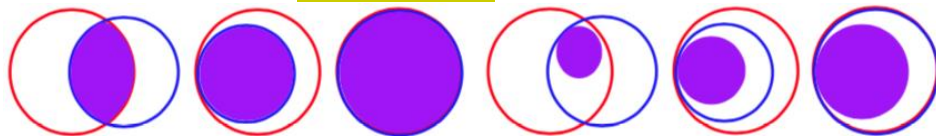
System	FOMs	2OMs	SS2OMs	<i>strict-</i>
Vending Machine	57	1,488	25	-
Triangle	138	9,086	393	5
Monopoly	866	372,885	3,324	58
Commons CSV	925	425,531	4,430	15
Commons CLI	1,082	582,685	1,852	41
ECal	1,207	727,556	1,421	26
Commons Validator	3,197	5,102,330	17,546	121
Gson	3,712	6,880,416	6,970	140
Chess	5,287	13,952,357	8,959	173
Joda-Time	16,333	133,394,201	36,178	374
Overall	32,804	161,394,201	81,098	953

RQ2: Distribution of mutations

System	SS2OMs			strict-SS2OMs			Pot. Subtle 2OMs		
	SM	SC	DC	SM	SC	DC	SM	SC	DC
Vending Machine	20	5	-	-	-	-	-	-	-
Triangle	393	-	-	5	-	-	13	-	-
Monopoly	1,110	1,470	596	13	3	42	30	-	16
Commons CSV	3,422	736	272	13	2	-	11	-	-
Commons CLI	1,627	199	26	41	-	-	8	-	-
ECal	761	538	122	22	4	-	-	-	-
Commons Validator	6,416	10,442	713	106	15	-	54	-	-
Gson	4,291	2,541	138	139	1	-	48	31	-
Chess	8,738	201	38	165	8	-	304	-	-
Joda-Time	25,932	7,907	2,339	394	25	-	118	16	-

86.42%

RQ3: Venn



Vending Machine	-	7	18	-	-	-
Triangle	38	143	207	-	4	1
Monopoli	1	414	2,851	1	2	55
Commons CSV	-	116	4,299	-	1	14
Commons CLI	9	429	1,373	-	30	11
ECal	8	223	1,164	-	11	15
Commons Validator	32	1,526	15,867	-	20	101
Gson	60	1,079	5,691	-	73	67
Chess	265	1,674	6,847	5	83	85
Joda-Time	31	2,322	33,451	-	70	304

Mutants reduction - example

FOMs

m1

m2

m3

m4

m5

m6

m7

m8

Mutants reduction - example

FOMs

SS2OMs

m1

[m1,m2]

m2

[m3,m4]

m3

[m5,m6]

m4

m5

m6

m7

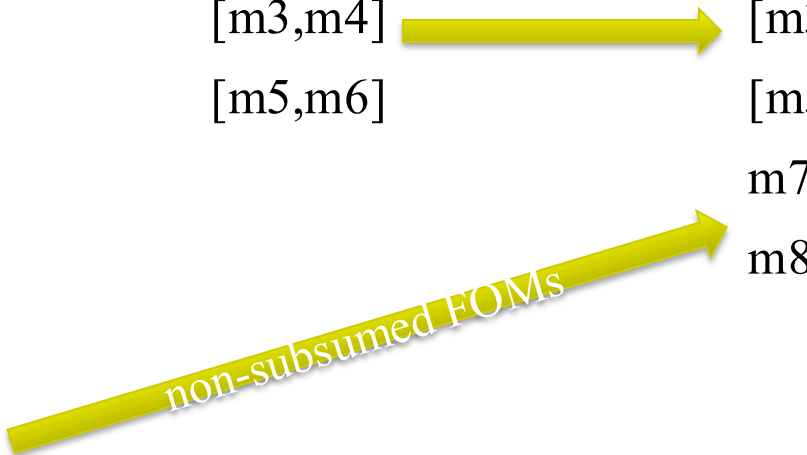
m8

Mutants reduction - example

FOMs	SS2OMs	Resulting mutants
m1	[m1,m2]	[m1,m2]
m2	[m3,m4]	[m3,m4]
m3	[m5,m6]	[m5,m6]
m4		
m5		
m6		
m7		
m8		

Mutants reduction - example

FOMs	SS2OMs	Resulting mutants
m1	[m1,m2]	[m1,m2]
m2	[m3,m4]	[m3,m4]
m3	[m5,m6]	[m5,m6]
m4		m7
m5		m8
m6		
m7		
m8		



non-subsumed FOMs

Mutants reduction - example

FOMs	SS2OMs	Resulting mutants	Reduction
m1	[m1,m2]	[m1,m2]	3 out of 8 mutants (37.5%)
m2	[m3,m4]	[m3,m4]	
m3	[m5,m6]	[m5,m6]	
m4		m7	
m5		m8	
m6			
m7			
m8			

non-subsumed FOMs

With much more SS2OMs

FOMs

SS2OMs

Resulting mutants

m1 [m1,m2], [m1,m4], ...,

m2 [m2,m3], ..., [m2,m8],

m3 [m3,m4],..., [m3,m8],

m4 [m4,m5], ..., [m4,m8],

m5 [m5,m6], [m5,m7], ...,

m6 [m6,m7], [m6,m8],

m7 [m7,m8], [m1,m7], ...,

m8 [m2,m4], ..., [m5,m8]

??????????

**Minimum
Set Cover
Problem**

RQ6: Mutants reduction (1/2)

System	SS2OMs	SS2OMs red.	SS2OMs discarded	Non-sub. FOMs	Resulting mutants
Vending Machine	25	17	32.00%	32	49
Triangle	393	52	86.77%	36	88
Monopoli	3,176	270	91.50%	387	657
Commons CSV	4,430	234	94.72%	503	737
Commons CLI	1,066	311	70.83%	545	856
ECal	1,488	366	75.40%	568	934
Commons Validator	17,571	944	94.63%	1,464	2,408
Gson	6,970	976	86.00%	1,978	2,954
Chess	8,977	1,402	84.38%	2,832	4,234
Joda-Time	36,178	2,531	93.00%	11,801	14,332
Overall	81,098	7,161	91.17%	19,951	27,112

RQ6: Mutants reduction (2/2)

System	FOMs	Resulting mutants	Mutants reduction
Vending Machine	57	49	14.04%
Triangle	138	88	36.23%
Monopoli	866	657	24.13%
Commons CSV	925	737	20.32%
Commons CLI	1,082	856	31.05%
ECal	1,213	934	22.54%
Commons Validator	3,197	2,408	24.52%
Gson	3,712	2,954	20.42%
Chess	5,319	4,234	20.38%
Joda-Time	16,333	14,332	12,25%
Overall	32,804	27,112	17.35%

Reaching Analysis

Motivation

We were interested on investigating

- What makes the SS2OMs harder to kill than their constituent FOMs?

In other words:

- How the SS2OMs mutations partially mask each other?
- While **inspecting** sampling source codes, we discovered:

**For some SS2OMs, their killing tests
do not reach both of their mutations**

SS2OMs *reaching* example

SS2OMs	Killing tests size	Tests reaching both mutations	Tests reaching mutation 1	Tests reaching mutation 2
SS2OM ₁	9	9	-	-
SS2OM ₂	10	-	8	2
SS2OM ₃	6	-	6	-
SS2OM ₄	4	-	-	4
SS2OM ₅	8	4	3	1
SS2OM ₆	7	4	-	3
SS2OM ₇	5	2	3	-

} can mask

} cannot mask

Research questions

- **RQ1:** What is the **proportion** of SS2OMs so that their mutations **can mask** each other?
- **RQ2:** reduction/masking (not presented)

SS2OMs *reaching* definitions

SS2OMs	Killing tests size	Tests reaching both mutations	Tests reaching mutation 1	Tests reaching mutation 2	
SS2OM ₁	9	9	-	-	} Reach _{max}
SS2OM ₂	10	-	8	2	
SS2OM ₃	6	-	6	-	} Reach _{min}
SS2OM ₄	4	-	-	4	
SS2OM ₅	8	4	3	1	} Reach _{int}
SS2OM ₆	7	4	-	3	
SS2OM ₇	5	2	3	-	

RQ1: SS2OMs *reaching*

System	SS2OMs	$Reach_{max} \%$	$Reach_{int} \%$	$Reach_{min} \%$
Vending Machine	25	12	8	80
Triangle	393	8	1	91
Monopoly	3,176	65	1	34
Commons CSV	4,430	80	1	19
Commons CLI	1,066	23	4	73
ECal	1,488	48	5	47
Commons Validator	17,571	55	4	41
Gson	6,970	47	3	50
Chess	8,977	48	2	50
Overall	44,920	53,52%	3,01%	43,47%

First + Second-Mutant Subsumption

Motivation example

test	m1	m2	m3	m4	m5
t1	×	×		×	×
t2	×		×	×	
t3				×	
t4		×		×	×

Subsumption relationships

test	m1	m2	m3	m4	m5
t1	×	×		×	×
t2	×		×	×	
t3				×	
t4		×		×	×

$m1 \rightarrow m4$

$m2 \rightarrow m4$

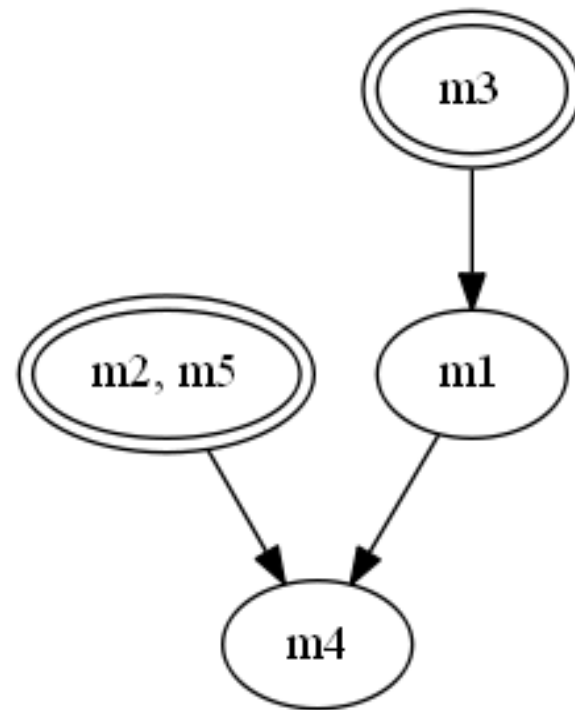
$m3 \rightarrow m1$

$m3 \rightarrow m4$

$m5 \rightarrow m4$

Subsumption Graph

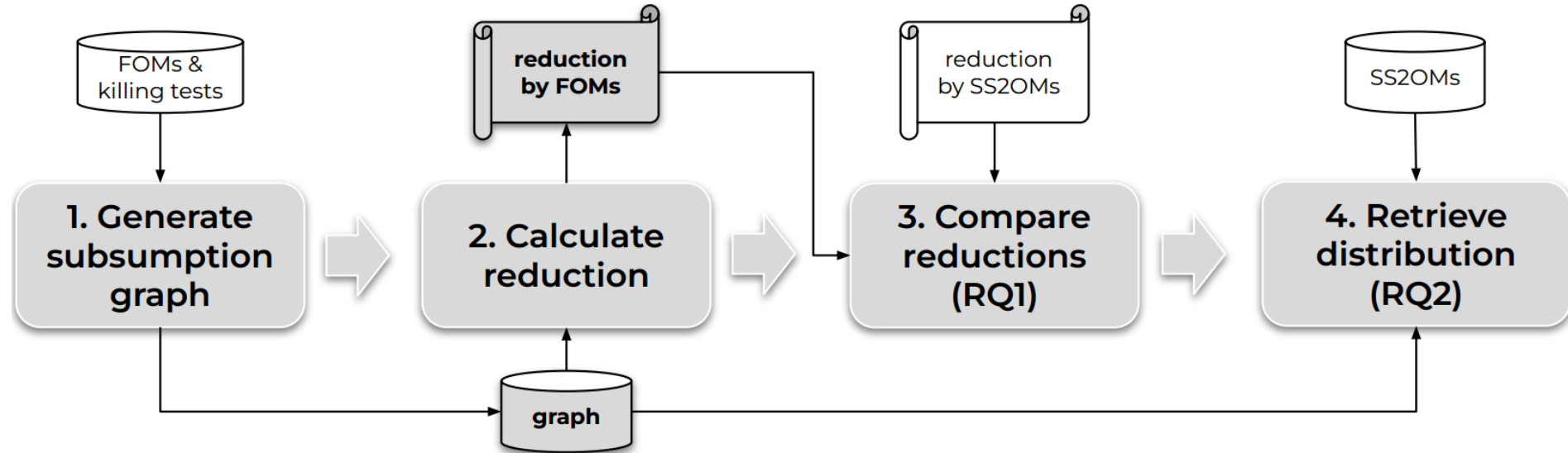
Test	m1	m2	m3	m4	m5
t1	×	×		×	×
t2	×		×	×	
t3				×	
t4		×		×	×



RQs

- **RQ1:** Which *subsumption* can reduce more mutants?
 - From FOMs
 - From SS2OMs
- **RQ2:** Can both be combined?

Study steps



Isolated reductions

System	FOMs	Via subsumption graph	Via SS2OMs
Vending Machine	57	66,67%	14.04%
Triangle	138	57,25%	36.23%
Monopoli	866	85,33%	24.13%
Commons CSV	925	71,89%	20.32%
Commons CLI	1,082	78,00%	31.05%
ECal	1,213	76,83%	22.54%
Commons Validator	3,197	73,01%	24.52%
Gson	3,712	76,40%	20.42%
Chess	5,319	80,86%	20.38%
Overall	16,471	77.35%	22.37%

Highlight on Triangle

{91}

{65, 67, 68, 70, 75, 77, 80, 52, 85, 56, 57, 58, 59, 63}

{35, 37, 38, 39, 108, 112, 114, 115, 116, 118, 119}

{129, 130, 131, 46, 122, 123, 124, 127}

{11}

{62}

{79}

{5}

{96, 132, 136, 121, 106, 111}

{69}

{97, 99, 100, 101, 103, 104, 23, 24, 25, 26, 27, 28, 93}

{0}

#FOMs	#minimal nodes	#remaining FOMs
138	12	59

RQ2 answer

SS2OMs do not seem to contribute to any further reduction than the one achieved by the subsuming FOMs.

Threats to validity and limitations

Threats to validity

❑ **External:**

- Java
- 10 systems
- 4 MutOps

❑ **Construction:** possible errors in our tool

❑ **Internal:**

- Only one large system
- Original test suites may have poor quality

Limitations

- ❑ Removal of some
 - Mutants
 - Test cases
- ❑ Significant time required to run for the largest systems

Related work

2nd order mutation testing

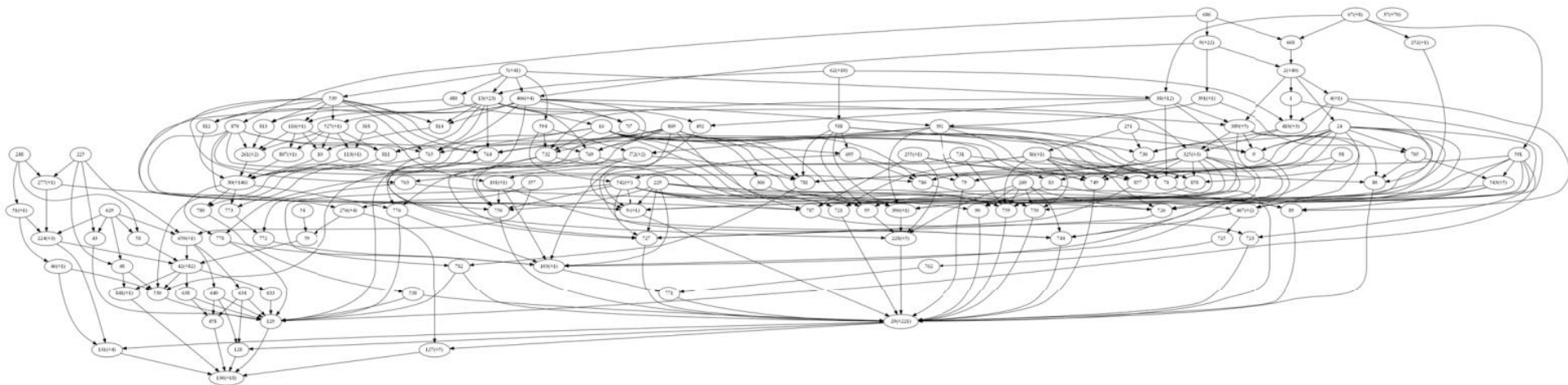
- 2OMs sampling
 - Random
 - LastToFirst
 - Different operators
 - etc.
- [Polo et al., 2009], [Kintis et al., 2010], [Madeisky et al., 2013] ...

SSHOMs

- [Jia et al., 2008 and 2009]
 - First search-based algorithms proposed
 - SSHOMs examples
- [Harman et al., 2014]
 - Reduction via greedy minimization
- [Wong et al., 2020]
 - Search for all possible SSHOMs via Variational Execution
 - Search_{pri} using some characteristics

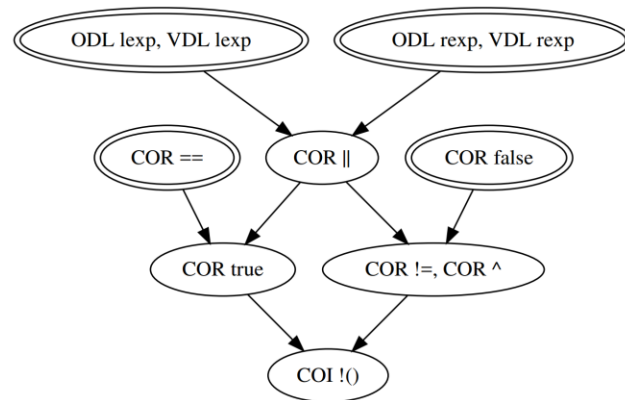
Kurtz *et al.* 2014

□ Mutant Subsumption Graphs



Guimarães *et al.* 2020

□ Optimizing Mutation Testing by Discovering Dynamic Mutant Subsumption Relations



Target	Minimal Set	Reduction
lexp + rexp	AORB %, ODL lexp, ODL rexp	62.5%
lexp + rexp (obj)	ODL lexp, ODL rexp	50.0%
lexp - rexp	AORB %, ODL lexp, ODL rexp	62.5%
lexp * rexp	AORB /, ODL lexp, ODL rexp	62.5%
lexp / rexp	AORB %, AOIS *, ODL rexp	62.5%
lexp % rexp	AORB +, AORB -, AORB /, ODL lexp	50.0%
lexp > rexp	ROR false, ROR !=, ROR >=	62.5%
lexp >= rexp	ROR true, ROR ==, ROR >	62.5%
lexp < rexp	ROR false, ROR !=, ROR <=	62.5%
lexp <= rexp	ROR true, ROR ==, ROR <	62.5%
lexp == rexp	ROR false, ROR <=, ROR >=	62.5%
lexp == rexp (obj)	ROR !=	50.0%
lexp == rexp (bool)	ROR !=, ODL lexp, ODL rexp	50.0%
lexp != rexp	ROR true, ROR <, ROR >	62.5%
lexp != rexp (obj)	ROR ==	50.0%
lexp != rexp (bool)	ROR ==, ODL lexp, ODL rexp	50.0%
lexp && rexp	COR false, COR ==	81.8%
lexp rexp	COR true, COR !=	81.8%
lexp & rexp	ODL lexp, ODL rexp	66.7%
lexp rexp	ODL lexp, ODL rexp, LOR ^	66.7%
lexp ^ rexp	LOR	83.3%
lexp ^ rexp (bool)	COR false, COR	80.0%
exp	AOIU -exp	83.3%
+exp	LOI exp	75.0%
!exp	COD exp	50.0%
-exp	AODU exp	66.7%
~exp	LOD exp	75.0%
	AODS exp	75.0%
	LOI exp	75.0%
	AODS exp	75.0%
	LOI exp	75.0%
	ASRS -=, ASRS %=, ODL =	50.0%
	ASRS +=, ASRS %=, ODL =	50.0%
	ASRS /=, ASRS %=, ODL =	50.0%
	ASRS *=, ASRS %=, ODL =	50.0%
	ASRS +=, ASRS -=, ASRS *=, ASRS /=	33.3%
	ASRS >>=,	66.7%
	ASRS <<=, ODL =, SDL	0.0%
	ASRS >>=, ASRS <<=	50.0%
	ODL =, SDL	50.0%
	ODL =, ASRS ^=, SDL	25.0%
	ASRS =	75.0%

Masking

- We did not find related works about **SSHOMs** reaching and masking
- Although there is about faults masking:
[Gopinath et al., 2017]

Reaching Analysis

- As far as we are concerned, our study was the first one!

Final Remarks

Conclusion

- ❑ There are more SS2OMs than necessary
- ❑ Most have the same killing tests of their constituent FOMs
- ❑ Only 53% of $Reach_{max}$
- ❑ Mutants reduction via SS2OMs:
 - 17,35%
 - n Does not improve the reduction via FOMs

Conclusion

- Are SS2OMs in fact useful?
- Does it worth keep researching this topic?

Future work

- ❑ New mutation operators
- ❑ New systems
- ❑ New analyses
- ❑ Investigate how automated source code generation could benefit from mutation testing

Final Remarks

3900

IEEE TRANSACTIONS ON SOFTWARE ENGINEERING, VOL. 48, NO. 10, OCTOBER 2022

Practical Mutation Testing at Scale: A view from Google

Goran Petrović^{ID}, Marko Ivanković, Gordon Fraser, and René Just^{ID}

Abstract—Mutation analysis assesses a test suite's adequacy by measuring its ability to detect small artificial faults, systematically seeded into the tested program. Mutation analysis is considered one of the strongest test-adequacy criteria. Mutation testing builds on top of mutation analysis and is a testing technique that uses mutants as test goals to create or improve a test suite. Mutation testing has long been considered intractable because the sheer number of mutants that can be created represents an insurmountable problem—both in terms of human and computational effort. This has hindered the adoption of mutation testing as an industry standard. For example, Google has a codebase of two billion lines of code and more than 150,000,000 tests are executed on a daily basis. The traditional approach to mutation testing does not scale to such an environment; even existing solutions to speed up mutation analysis are insufficient to make it computationally feasible at such a scale. To address these challenges, this paper presents a scalable approach to mutation testing based on the following main ideas: (1) mutation testing is done incrementally, mutating only *changed code* during code review, rather than the entire code base; (2) mutants are filtered, removing mutants that are likely to be irrelevant to developers, and limiting the number of mutants per line and per code review process; (3) mutants are selected based on the historical performance of mutation operators, further eliminating irrelevant mutants and improving mutant quality. This paper empirically validates the proposed approach by analyzing its effectiveness in a code-review-based setting, used by more than 24,000 developers on more than 1,000 projects. The results show that the proposed approach produces orders of magnitude fewer mutants and that context-based mutant filtering and selection improve mutant quality and actionability. Overall, the proposed approach represents a mutation testing framework that seamlessly integrates into the software development workflow and is applicable to industrial settings of any size.

Index Terms—Mutation testing, code coverage, test efficacy

Final Remarks



Uncle Bob Martin ✓

@unclebobmartin



I'm significantly older than you. I started coding in the late 60s. My current strategy is to not read any of the code written by my agents. That's the only way I can take advantage of their productivity. What I do instead is to surround the agents with extreme constraints. Unit tests, gherkin tests, QA procedures, quality metrics, **mutation testing**, test coverage, and a plethora of others. In the end, I have very high confidence in the code they produce because they've had to run the gauntlet of all of my constraints and tests.

8:44 AM · Jul 23, 2026 · **5M** Views



568



1.9K



18K



13K



<https://x.com/unclebobmartin/status/2044114698451476492>



Questions?

